TRAINOCATE

Module 1

Linux fundamentals and incident triage

One day • 390 instructional minutes

Investigate → explain → mitigate → verify → hand over

TRAINOCATE

Today’s route

40 min | Access, identity and user creation
60 min | Navigation, files and package installation
45 min | Allocate space and diagnose disk limits
75 min | Command investigations and first Bash script
50 min | Install and inspect the application stack
35 min | Service failures and application log diagnosis
25 min | Resource and network troubleshooting
40 min | Linux incident challenge with your evidence script
20 min | Checkpoints and recovery reserve

The lab system and its boundaries

LEARNER

SSH + individual account

Team workspace

APPLICATION HOST

Tomcat + MySQL

Loyalty training app

DATABASE LAB

SQL Server 2022

Query Store + Agent

Request IDs connect application evidence.

Synthetic packs cover replication, IIS, SELinux and pooled-app incidents.

Fault injection and storage administration are instructor-controlled.

How to work an incident

1. State impact and capture UTC time.
2. Preserve evidence before changing the system.
3. Test a hypothesis with a discriminating check.
4. Make one scoped change with a rollback.
5. Verify the affected business function and communicate.

TRAINOCATE

Access, identity and user creation

You are the new support team. Establish individual identities before changing the shared loyalty server.

- The SSH key authenticates a login; it does not make the login root.
- A user, primary group, supplementary groups and home-directory permissions are separate controls.
- Use sudo only for the supervised administrative step; keep routine investigation unprivileged.

TRAINOCATE

Which identity can traverse this path?

File: rw-r----- (640) → **Owner rw; group r** → **Parents need search (x)**

Check every parent directory before changing the file mode.

Generate your own SSH key

```
ssh-keygen -t ed25519 -f "$HOME/.ssh/support_lab_ed25519"
cat "$HOME/.ssh/support_lab_ed25519.pub"
```

Read the complete worksheet before executing. Instructor-only changes stay supervised.

Prove it — and challenge the obvious answer

- ✓ Learner logs in using their own key and has a writable home.
- ✓ Private file has mode 600; team membership appears after a new login.

? Does a # character inside a command prove you are root?

? Why might id in an old shell omit a newly granted group?

? Does chmod 777 fix a missing filesystem quota?

Debrief and deliverable

Identity baseline, permission evidence and a short access checklist.

What did the evidence prove?

What alternative did you rule out?

How did you verify recovery?

What remains uncertain?

TRAINOCATE

Navigation, files and package installation

A deployment arrives as an archive. Find the right directory, inspect it, copy it safely and install diagnostic tools.

- Absolute paths start at /; relative paths start at the current working directory.
- Quote paths and distinguish copying a file from overwriting an existing destination.
- apt is the Ubuntu package manager; dnf examples are Red Hat comparisons, not commands for this host.

Your investigation sequence

1. Create and navigate your workspace
2. Create, copy and rename a file
3. Inspect packages and command lookup
4. Check the tools as a learner
5. Compress a copy and observe a broken link
6. Transfer the method without copying

Create and navigate your workspace

```
mkdir -p ~/support-lab/files-practice/{incoming,work,archive}
cd ~/support-lab/files-practice
pwd
ls -lah
cd /var/log
pwd
cd -
```

Read the complete worksheet before executing. Instructor-only changes stay supervised.

Prove it — and challenge the obvious answer

✓ Student explains why the symlink breaks after its target is removed.

✓ Commands are found through PATH; package installation is verified rather than assumed.

? Does cd /var/log change another terminal’s working directory?

? Why is sudo echo text > /root/file different from echo text | sudo tee /root/file?

? What happens when an unquoted variable contains spaces?

Debrief and deliverable

A directory tree, package verification table and safe file-handling notes.

What did the evidence prove?

What alternative did you rule out?

How did you verify recovery?

What remains uncertain?

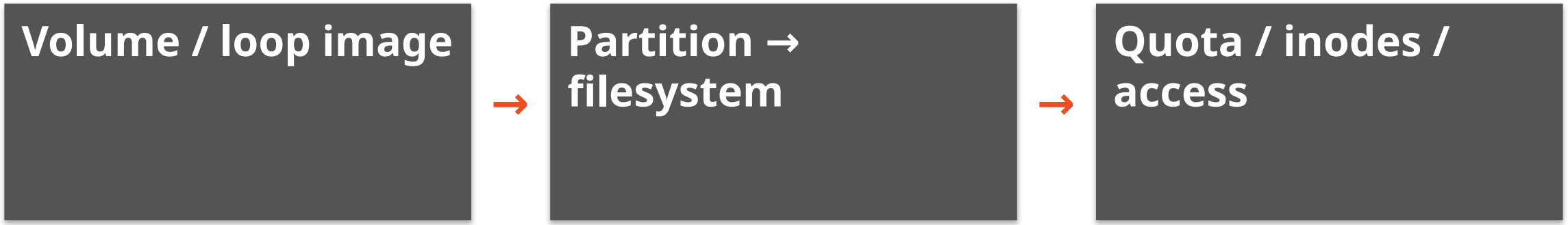
TRAINOCATE

Allocate space and diagnose disk limits

A learner reports “my EC2 user is capped.” Determine which layer is limiting writes before selecting a fix.

- EBS volume → partition → filesystem → directory/account; growing one layer does not automatically grow all layers.
- Compare byte capacity, inode capacity, quotas, permissions and deleted-open files.
- The observed host already has an expanded root filesystem; reproduce growth on a disposable disk.

Locate the storage limit



A larger device does not automatically enlarge its filesystem.

Record the actual limiting layer

```
id
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS
findmnt -T "$HOME"
df -hT "$HOME"
df -i "$HOME"
quota -s
```

Read the complete worksheet before executing. Instructor-only changes stay supervised.

Prove it — and challenge the obvious answer

✓ Student identifies the limiting layer from evidence, not the wording of the complaint.

✓ Disposable filesystem grows and can be cleaned up; root partition is untouched.

? df says full but du is small: name three explanations.

? Will adding a user allocate a new partition?

? Why can root sometimes write when an ordinary user cannot, even without quotas?

Debrief and deliverable

A storage decision tree and before/after capacity evidence.

What did the evidence prove?

What alternative did you rule out?

How did you verify recovery?

What remains uncertain?

TRAINOCATE

Command investigations and first Bash script

Operations received mixed logs and filenames. Produce accurate incident counts without losing records or matching misleading text.

- grep selects lines; awk interprets fields; sed transforms text; each has different quoting rules.
- Sort before uniq, choose numeric sorting for numbers and preserve filenames with NUL separators.
- Counts require a clear unit: lines, requests, users and incidents are not interchangeable.
- Create and execute a small Bash evidence collector before the incident challenge.

Your investigation sequence

1. Inspect the evidence without editing it
2. Compare a naive search with parsed status
3. Extract and transform fields
4. Handle unusual filenames safely
5. Observe an expected nonzero pipeline result
6. Build your first evidence script

Inspect and execute your saved script

```
cat "$HOME/support-lab/bin/triage.sh"
if bash -n "$HOME/support-lab/bin/triage.sh"; then
    cd "$HOME"
    ./support-lab/bin/triage.sh support-tomcat
    printf 'collector_exit=%s\n' "$?"
fi
```

Read the complete worksheet before executing. Instructor-only changes stay supervised.

Prove it — and challenge the obvious answer

- ✓ Counts match parsed fixture records; malformed rows are reported separately.
- ✓ All unusual filenames survive the archive round trip.
- ✓ Run triage.sh with two service arguments and explain the captured statuses.

? Does grep returning 1 always mean an error?

? Will uniq find nonadjacent duplicates?

? Can cut reliably parse arbitrary CSV with quoted commas?

Debrief and deliverable

Pipelines, archive check and triage.sh with two-input test evidence.

What did the evidence prove?

What alternative did you rule out?

How did you verify recovery?

What remains uncertain?

Install and inspect the application stack

Deploy the loyalty application, establish a healthy baseline and understand the service before introducing failures.

- A Java runtime, servlet container, database and application configuration are separate dependencies.
- systemd unit configuration defines process identity, environment, startup and restart behavior.
- A live process is weaker evidence than an authenticated business request.

Your investigation sequence

1. Inspect the setup stages
2. Install the shared stack
3. Repair a copied service configuration
4. Inspect service identity and logs
5. Verify readiness and business data
6. Open a browser through SSH forwarding

Repair a copied service configuration

```
practice_dir=$(mktemp -d "$HOME/support-lab/unit-practice-XXXXXX")
cp "$COURSE_ROOT/shared/templates/service-practice.service" "$practice_dir/support-tomcat.service"
systemctl cat support-tomcat
nano "$practice_dir/support-tomcat.service"
cat "$practice_dir/support-tomcat.service"
```

Read the complete worksheet before executing. Instructor-only changes stay supervised.

Prove it — and challenge the obvious answer

✓ Readiness returns 200 and player 1 returns synthetic data.

✓ Student explains why Tomcat runs as supportapp and MySQL is not exposed publicly.

? Does systemctl enable start a stopped service?

? Can /live return 200 while the database password is wrong?

? Why should you collect the journal before restart?

Debrief and deliverable

A component diagram and a healthy baseline evidence bundle.

What did the evidence prove?

What alternative did you rule out?

How did you verify recovery?

What remains uncertain?

Service failures and application log diagnosis

The application worked before a change. One team investigates a startup fault; another receives a permission fault after reset.

- Capture unit state, exit status and journal before changing configuration.
- Correlate an HTTP response request ID with its application log line.
- Distinguish startup failure, runtime failure and failed dependency checks.

Your investigation sequence

1. Capture the healthy baseline
2. Introduce and observe a startup fault
3. Locate the failing startup configuration
4. Reset, then introduce a different failure
5. Inspect runtime symptoms and file access
6. Verify permissions, restore and retest

Capture the healthy baseline

```
date -u +%FT%TZ
curl -i http://127.0.0.1:8080/loyalty/health
```

Read the complete worksheet before executing. Instructor-only changes stay supervised.

Prove it — and challenge the obvious answer

- ✓ Evidence names the actual failed executable or unreadable file.
- ✓ Recovery includes HTTP and business checks, not only active service status.

? Does a restart policy repair a bad executable path?

? Does a missing ERROR line prove the service started?

? Why is chmod 777 an inappropriate application-config fix?

Debrief and deliverable

A timeline with failing and recovered request IDs.

What did the evidence prove?

What alternative did you rule out?

How did you verify recovery?

What remains uncertain?

TRAINOCATE

Resource and network troubleshooting

Users report intermittent errors. Determine whether CPU, memory, DNS, TCP, TLS or database authentication evidence supports the complaint.

- High load is not the same measurement as CPU utilization; memory available differs from free.
- A listening port and successful TCP connection do not prove database authentication.
- Ubuntu firewall checks differ from Red Hat SELinux indicators; do not disable controls as a diagnostic shortcut.

Your investigation sequence

1. Measure resources over an interval
2. Observe bounded CPU and file handles
3. Test network layers in order
4. Compare TCP with authenticated readiness
5. Analyze platform evidence
6. Transfer the method without copying

Measure resources over an interval

```
uptime
free -h
ps -eo pid,ppid,user,comm,pcpu,pmem --sort=-pcpu | head
top -b -n 1 | head -n 20
pidstat 1 3
```

Read the complete worksheet before executing. Instructor-only changes stay supervised.

Prove it — and challenge the obvious answer

- ✓ Student differentiates TCP reachability from successful authenticated queries.
- ✓ Diagnosis cites interval evidence and acknowledges ICMP/filtering limitations.

? Does failed ping prove the host is down?

? Is low free memory alone proof of memory pressure?

? Does curl -k repair an expired certificate?

Debrief and deliverable

A layered connectivity table with evidence and remaining unknowns.

What did the evidence prove?

What alternative did you rule out?

How did you verify recovery?

What remains uncertain?

Linux incident challenge with your evidence script

A deployment left the loyalty application unavailable. Your team has 25 minutes to identify the fault and deliver a reusable evidence collector.

- Build a small script from understood commands, then test failure behavior.
- Record command failures without losing the rest of the incident evidence.
- A recovery claim needs a customer-facing check and a clear incident summary.

Your investigation sequence

1. Prepare your submission workspace
2. Extend your existing evidence script
3. Receive the incident and assign roles
4. Investigate within the time budget
5. Collect and inspect evidence
6. Propose one mitigation and define recovery

Prepare your submission workspace

```
cp -i "$COURSE_ROOT/shared/templates/incident-report.md"
"$HOME/support-lab/evidence/m1-08/incident.md"
date -u +%FT%TZ
nano "$HOME/support-lab/evidence/m1-08/incident.md"
```

Read the complete worksheet before executing. Instructor-only changes stay supervised.

Prove it — and challenge the obvious answer

✓ Script runs without interactive input and records UTC timestamps.

✓ Incident report includes evidence, probable cause, mitigation and recovery validation.

? Why can an evidence collector legitimately exit nonzero and still produce a useful bundle?

? Does chmod +x guarantee a script will run on every shell?

Debrief and deliverable

Executable triage script, checklist, evidence bundle and incident report.

What did the evidence prove?

What alternative did you rule out?

How did you verify recovery?

What remains uncertain?

Advanced Linux command challenge bank

You must produce defensible answers from awkward input without changing source evidence.

- Predict command behavior before running it.
- Preserve evidence and explain assumptions about input format.
- Robust pipelines handle empty input, malformed records and unusual filenames.

Your investigation sequence

1. Create an isolated challenge workspace
2. Prepare an answer notebook
3. Solve and test counterexamples
4. Review with another learner

Create an isolated challenge workspace

```
mkdir -p ~/support-lab/challenges
challenge_dir=$(mktemp -d "$HOME/support-lab/challenges/commands-XXXXXX")
cp -a "$COURSE_ROOT/shared/fixtures/." "$challenge_dir/"
cd "$challenge_dir"
pwd
```

Read the complete worksheet before executing. Instructor-only changes stay supervised.

TRAINOCATE

Prove it — and challenge the obvious answer

- ✓ Each answer includes an explanation and a counterexample where appropriate.
- ✓ No command mutates original evidence or uses blanket deletion.

? Is line count always record count?

? When does tail -f miss a replacement log file?

? Can grep ERROR count HTTP failures accurately?

Debrief and deliverable

A reviewed command notebook; extensions do not displace required class activities.

What did the evidence prove?

What alternative did you rule out?

How did you verify recovery?

What remains uncertain?

Reference shelf and completion checks

Use shared/SOURCES.md for official technical references.

Submit scripts, evidence, interpretations and recovery checks.

Keep private keys and passwords out of submissions.

Complete the core labs before attempting extension challenges.

Evidence 25 • Diagnosis 25 • Recovery 20 • Automation 20 • Communication 10